

Chapter 2: Application layer

2.1 Principles of network applications

2.2 Web and HTTP

2.3 FTP

2.4 Electronic Mail
SMTP, POP3, IMAP

2.5 DNS

2.6 P2P applications

2.7 Socket programming
with TCP

2.8 Socket programming
with UDP

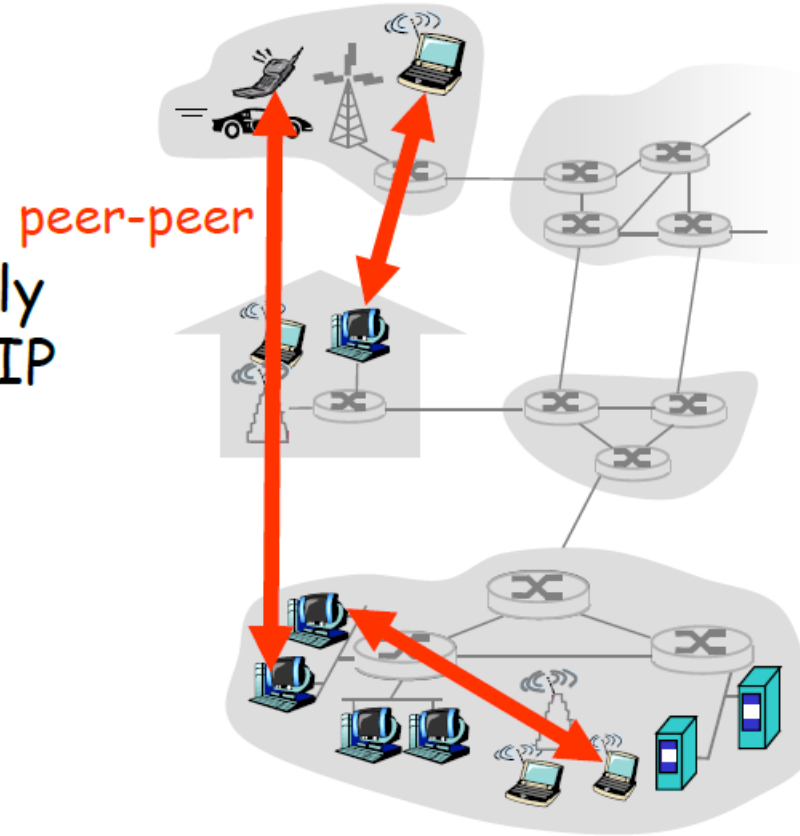
Application architectures

- ❖ client-server
- ❖ peer-to-peer (P2P)
- ❖ hybrid of client-server and P2P

Pure P2P architecture

- ❖ *no* always-on server
- ❖ arbitrary end systems directly communicate
- ❖ peers are intermittently connected and change IP addresses

highly scalable but
difficult to manage



Hybrid of client-server and P2P

Skype

- voice-over-IP P2P application
- centralized server: finding address of remote party:
- client-client connection: direct (not through server)

Instant messaging

- chatting between two users is P2P
- centralized service: client presence detection/location
 - user registers its IP address with central server when it comes online
 - user contacts central server to find IP addresses of buddies

Processes communicating

process: program running within a host.

- ❖ within same host, two processes communicate using **inter-process communication** (defined by OS).
- ❖ processes in different hosts communicate by exchanging **messages**

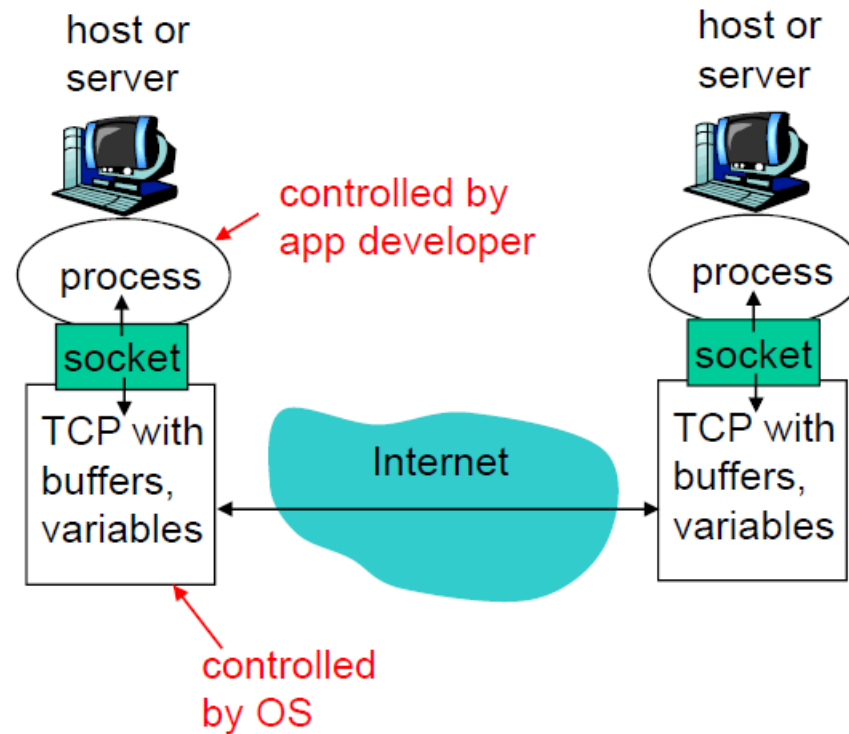
client process: process that initiates communication

server process: process that waits to be contacted

- ❖ aside: applications with P2P architectures have client processes & server processes

Sockets

- ❖ process sends/receives messages to/from its **socket**
- ❖ socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door which brings message to socket at receiving process



Addressing processes

- ❖ to receive messages, process must have *identifier*
- ❖ host device has unique 32-bit IP address
- ❖ Q: does IP address of host on which process runs suffice for identifying the process?
 - A: No, many processes can be running on same host
- ❖ *identifier* includes both IP address and port numbers associated with process on host.
- ❖ example port numbers:
 - HTTP server: 80
 - Mail server: 25
- ❖ to send HTTP message to gaia.cs.umass.edu web server:
 - IP address: 128.119.245.12
 - Port number: 80

App-layer protocol defines

- ❖ types of messages exchanged,
 - e.g., request, response
- ❖ message syntax:
 - what fields in messages & how fields are delineated
- ❖ message semantics
 - meaning of information in fields
- ❖ rules for when and how processes send & respond to messages

public-domain protocols:

- ❖ defined in RFCs
- ❖ allows for interoperability
- ❖ e.g., HTTP, SMTP

proprietary protocols:

- ❖ e.g., Skype

What transport service does an app need?

Data loss

- ❖ some apps (e.g., audio) can tolerate some loss
- ❖ other apps (e.g., file transfer, telnet) require 100% reliable data transfer

Timing

- ❖ some apps (e.g., Internet telephony, interactive games) require low delay to be "effective"

Throughput

- ❖ some apps (e.g., multimedia) require minimum amount of throughput to be "effective"
- ❖ other apps ("elastic apps") make use of whatever throughput they get

Security

- ❖ encryption, data integrity, ...

Internet transport protocols services

TCP service:

- ❖ *connection-oriented*: setup required between client and server processes
- ❖ *reliable transport* between sending and receiving process
- ❖ *flow control*: sender won't overwhelm receiver
- ❖ *congestion control*: throttle sender when network overloaded
- ❖ *does not provide*: timing, minimum throughput guarantees, security

UDP service:

- ❖ unreliable data transfer between sending and receiving process
- ❖ does not provide: connection setup, reliability, flow control, congestion control, timing, throughput guarantee, or security

Q: why bother? Why is there a UDP?

Internet apps: application, transport protocols

<u>Application</u>	<u>Application layer protocol</u>	<u>Underlying transport protocol</u>
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
Internet telephony	SIP, RTP, proprietary (e.g., Skype)	typically UDP

Chapter 2: Application layer

2.1 Principles of network applications

- app architectures
- app requirements

2.2 Web and HTTP

2.3 FTP

2.4 Electronic Mail

- SMTP, POP3, IMAP

2.5 DNS

2.6 P2P applications

2.7 Socket programming with TCP

2.8 Socket programming with UDP

Web and HTTP

First, a review...

- ❖ **web page** consists of **objects**
- ❖ object can be HTML file, JPEG image, Java applet, audio file,...
- ❖ web page consists of **base HTML-file** which includes several referenced objects
- ❖ each object is addressable by a **URL**
- ❖ example URL:

`www.someschool.edu/someDept/pic.gif`

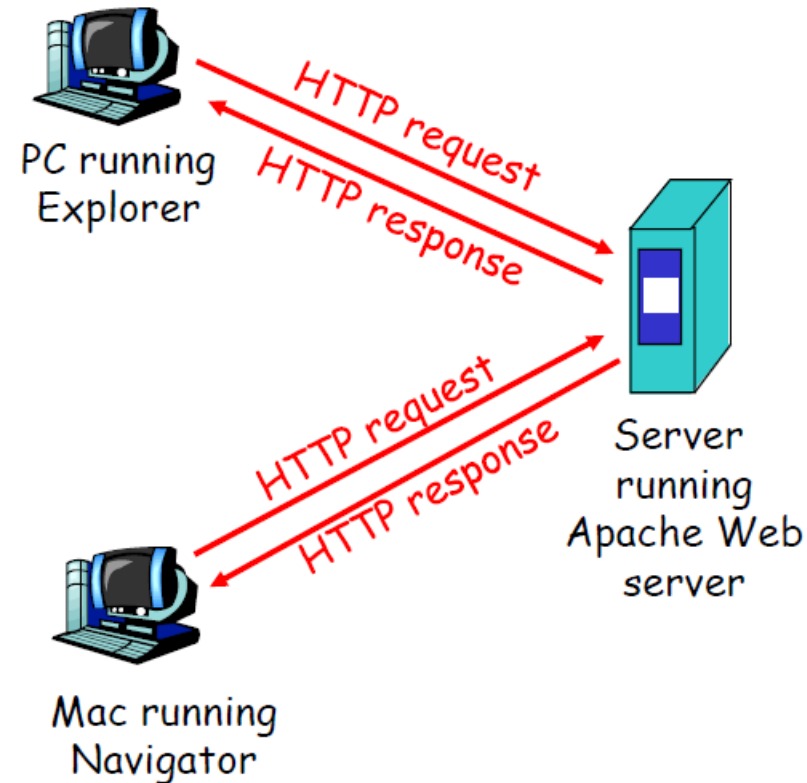
host name

path name

HTTP overview

HTTP: hypertext transfer protocol

- ❖ Web's application layer protocol
- ❖ client/server model
 - *client*: browser that requests, receives, "displays" Web objects
 - *server*: Web server sends objects in response to requests



HTTP overview (continued)

Uses TCP:

- ❖ client initiates TCP connection (creates socket) to server, port 80
- ❖ server accepts TCP connection from client
- ❖ HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- ❖ TCP connection closed

HTTP is "stateless"

- ❖ server maintains no information about past client requests

aside
protocols that maintain "state" are complex!

- ❖ past history (state) must be maintained
- ❖ if server/client crashes, their views of "state" may be inconsistent, must be reconciled

HTTP connections

non-persistent HTTP

- ❖ at most one object sent over TCP connection.

persistent HTTP

- ❖ multiple objects can be sent over single TCP connection between client, server.

Non-Persistent HTTP Connection Example (HTTP/1.0)

Scenario: A webpage with 1 HTML file and 10 images.

Process:

- The browser opens a connection, downloads the HTML, closes the connection.
- Then it opens another connection for the first image, downloads it, closes.
- Repeat this for all 10 images, requiring 11 separate TCP connections (11 open/close cycles).

Key Feature: One object per TCP connection.

Persistent HTTP Connection Example (HTTP/1.1)

Scenario: Same webpage with 1 HTML file and 10 images.

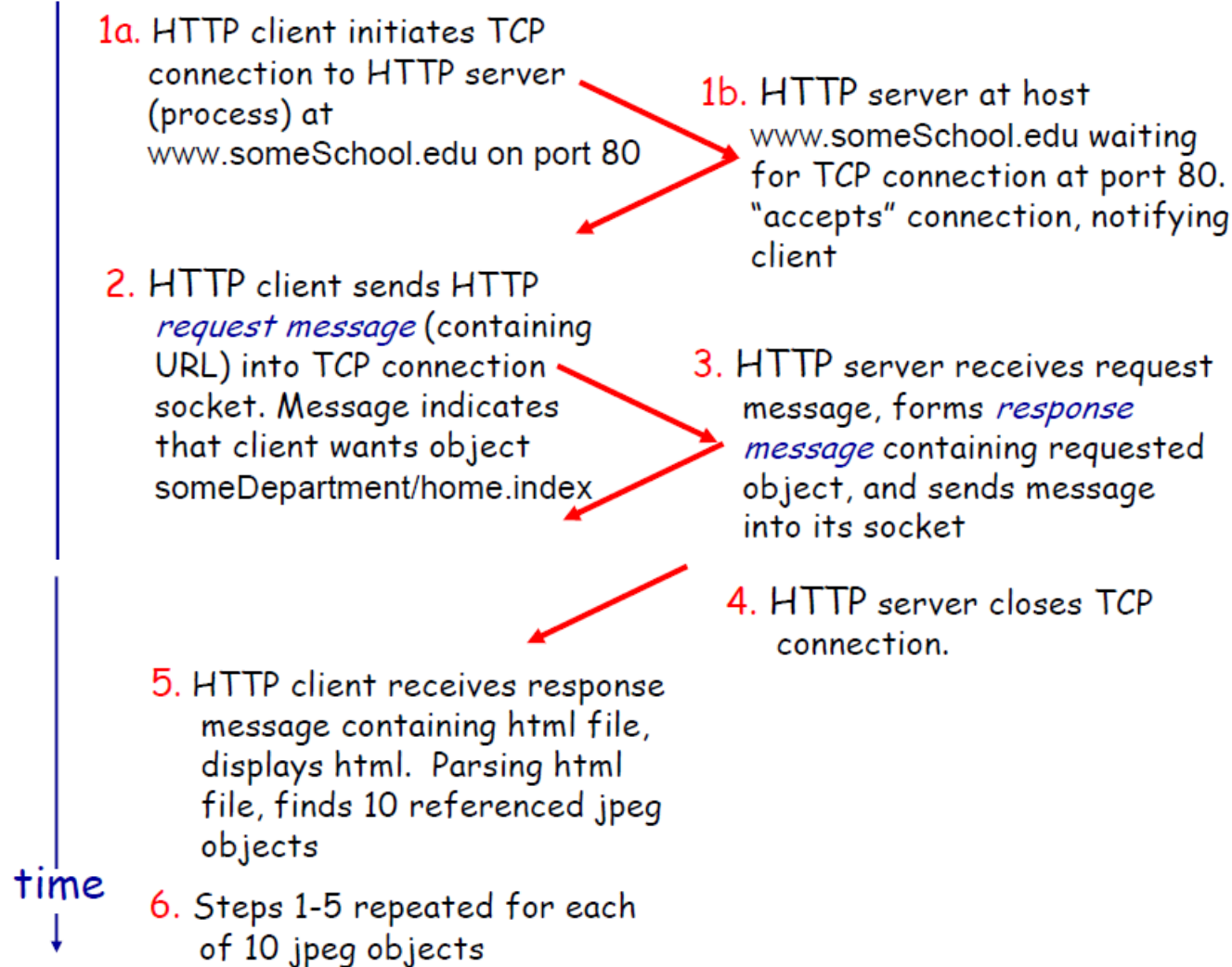
Process: The browser opens one TCP connection, downloads the HTML, then downloads all 10 images sequentially (or pipelined) over that same connection before it closes.

Key Feature: Multiple objects over a single TCP connection.

Nonpersistent HTTP

suppose user enters URL:

`www.someSchool.edu/someDepartment/home.index` (contains text,
references to 10
jpeg images)



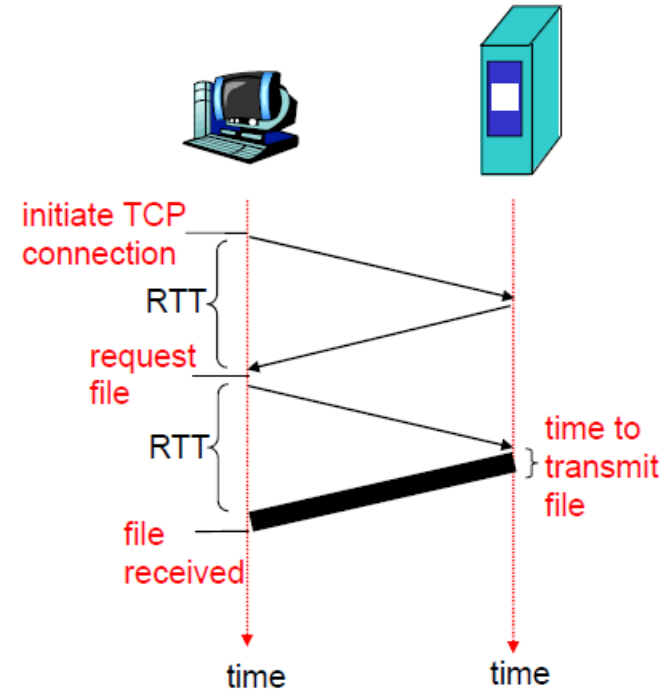
Non-Persistent HTTP: Response time

definition of RTT: time for a small packet to travel from client to server and back.

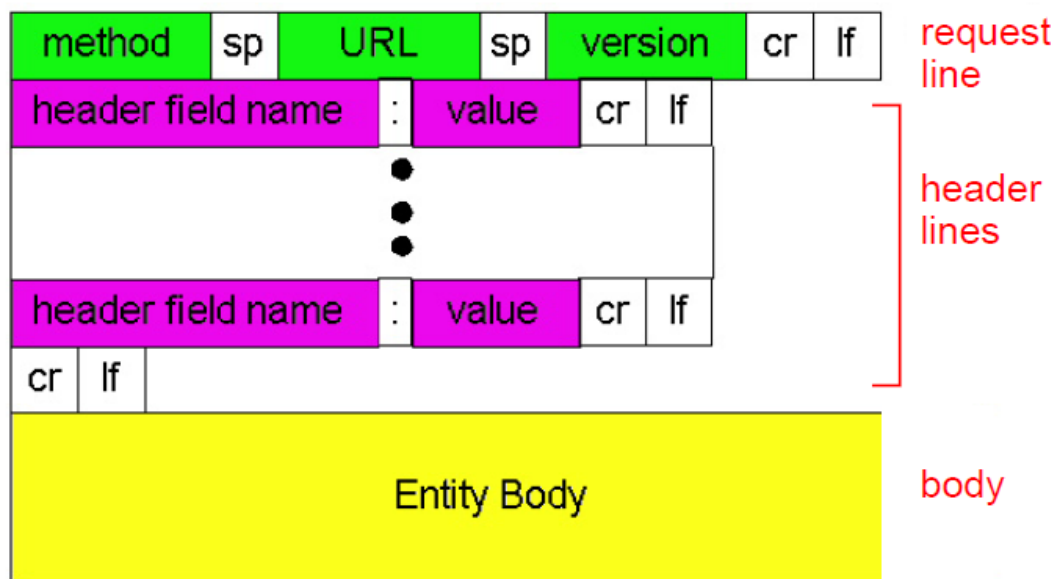
response time:

- ❖ one RTT to initiate TCP connection
- ❖ one RTT for HTTP request and first few bytes of HTTP response to return
- ❖ file transmission time

total = $2RTT + \text{transmit time}$



HTTP request message: general format



Field	Purpose
Method	Action to be performed
URL	Resource location
Version	HTTP protocol version
Header Lines	Metadata about request
Blank Line	Separates headers and body
Entity Body	Data sent to server

Method types

Method

- Specifies the **action** to be performed on the resource.

HTTP/1.0

- ❖ GET
- ❖ POST
- ❖ HEAD
 - asks server to leave requested object out of response

HTTP/1.1

- ❖ GET, POST, HEAD
- ❖ PUT
 - uploads file in entity body to path specified in URL field
- ❖ DELETE
 - deletes file specified in the URL field

- **GET** – Request data from the server
- **POST** – Send data to server
- **PUT** – Update a resource
- **DELETE** – Remove a resource
- **HEAD** – Request headers only

HTTP response message

status line
(protocol
status code
status phrase)

header
lines

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02
GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
1\r\n
\r\n
data data data data data ...
```

data, e.g.,
requested
HTML file

a) Protocol Version

- Indicates the **HTTP version** used by the server to respond.
- Ensures compatibility between client and server.

b) Status Code

- A **3-digit numeric code** that indicates the **result of the client's request**.

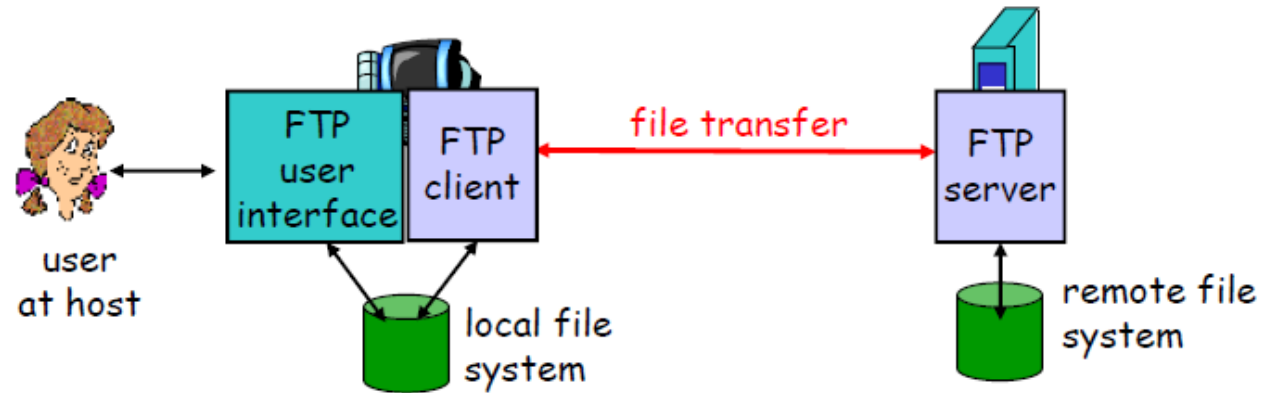
Common status codes:

- **200** – OK (request successful)
- **301** – Moved Permanently
- **400** – Bad Request
- **404** – Not Found
- **500** – Internal Server Error

c) Status Phrase

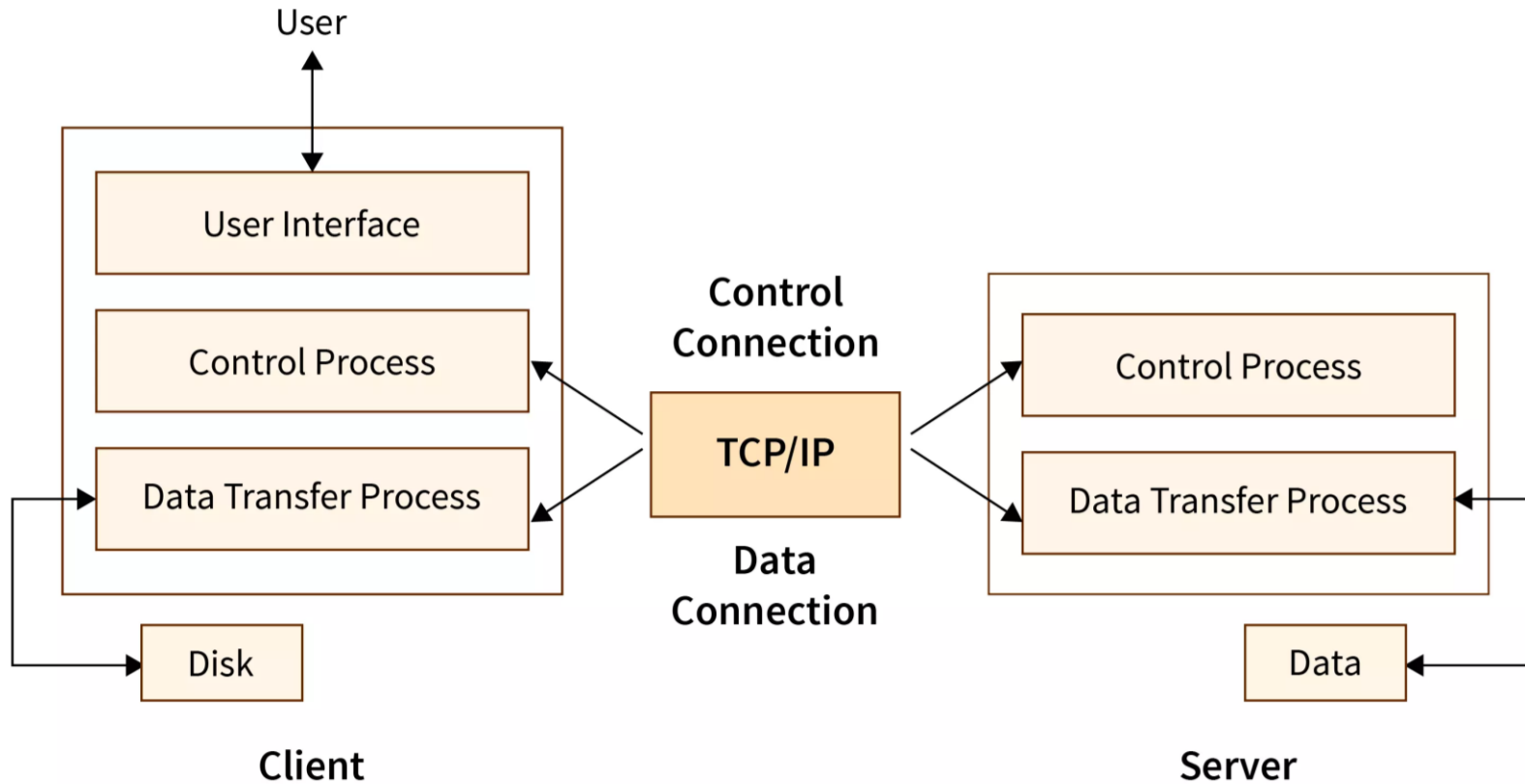
- A **human-readable description** of the status code.
- Helps in understanding the response easily.

FTP: the file transfer protocol



- ❖ transfer file to/from remote host
- ❖ client/server model
 - *client*: side that initiates transfer (either to/from remote)
 - *server*: remote host
- ❖ ftp: RFC 959
- ❖ ftp server: port 21

FTP: the file transfer protocol



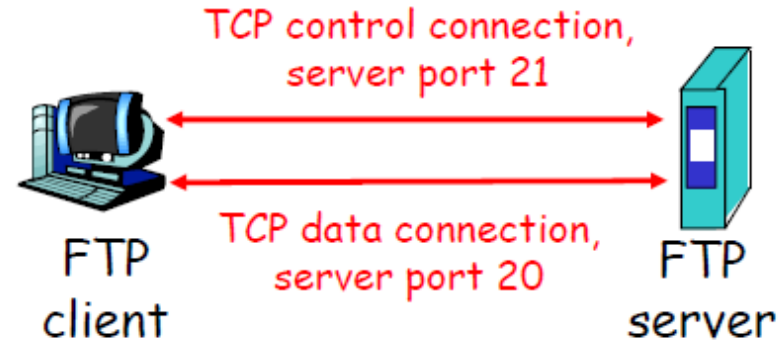
TCP/IP

Provides **reliable, connection-oriented communication**.

Client Side	Server Side
a) User - The end user who interacts with the system. - Issues commands such as login, upload, download, list files.	a) Control Process Receives commands from the client. Interprets and executes FTP commands. Sends response codes back to the client.
b) User Interface - Provides an interface (command-line or GUI) - Accepts FTP commands like GET, PUT, LS, QUIT.	b) Data Transfer Process Handles file upload/download operations. Communicates directly with the client's data transfer process.
c) Control Process - Manages the FTP control connection. - Sends commands from the client to the server. - Receives responses from the server. - Remains active throughout the session.	
d) Data Transfer Process - Handles the actual file transfer. - Sends or receives data through a separate data connection.	

FTP: separate control, data connections

- ❖ FTP client contacts FTP server at port 21, TCP is transport protocol
- ❖ client authorized over control connection
- ❖ client browses remote directory by sending commands over control connection.
- ❖ when server receives file transfer command, server opens 2nd TCP connection (for file) to client
- ❖ after transferring one file, server closes data connection.



- ❖ server opens another TCP data connection to transfer another file.
- ❖ control connection: "out of band" 🟡
- ❖ FTP server maintains "state": current directory, earlier authentication

How a Client Browses a Remote Directory Using the Control Connection (FTP)

In FTP, the **client browses the remote directory by sending directory-related commands over the control connection** established between the client and server.

1. After establishing the **control connection** (TCP port 21), the client sends FTP commands such as:
 - **PWD** – to display the present working directory
 - **CWD** – to change the working directory
 - **LIST** or **NLST** – to list files and directories
2. These commands are sent from the **client control process** to the **server control process** through the **control connection**.
3. The server interprets the commands and sends **status responses** (e.g., 200, 250) back to the client over the same control connection.
4. When a directory listing is requested (**LIST**), the server opens a **separate data connection** and sends the directory contents through it, while the command itself and responses continue to use the **control connection**.
5. The client displays the received directory information to the user via the **user interface**, allowing the user to browse the remote directory.

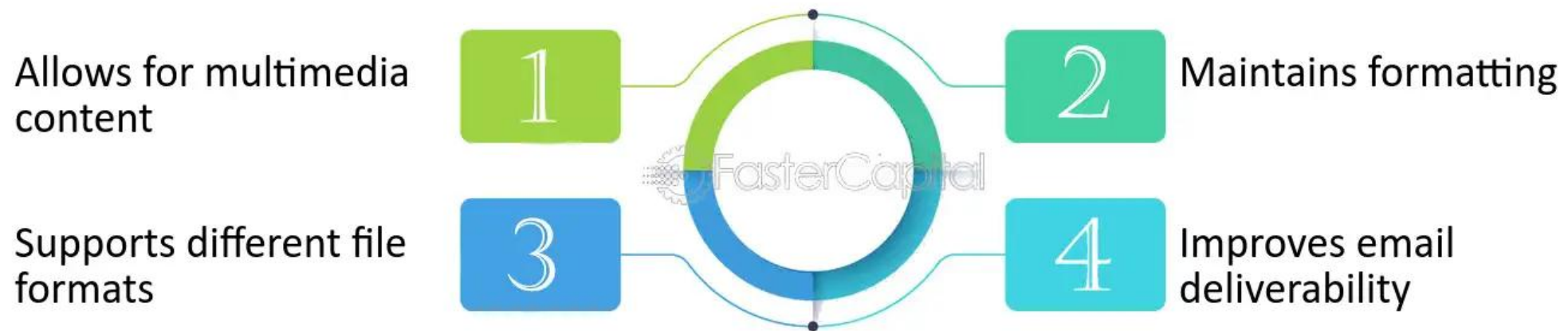
An **FTP server is a stateful protocol**, meaning it **maintains information (state) about each client session** throughout the connection.

The FTP server maintains the following state information:

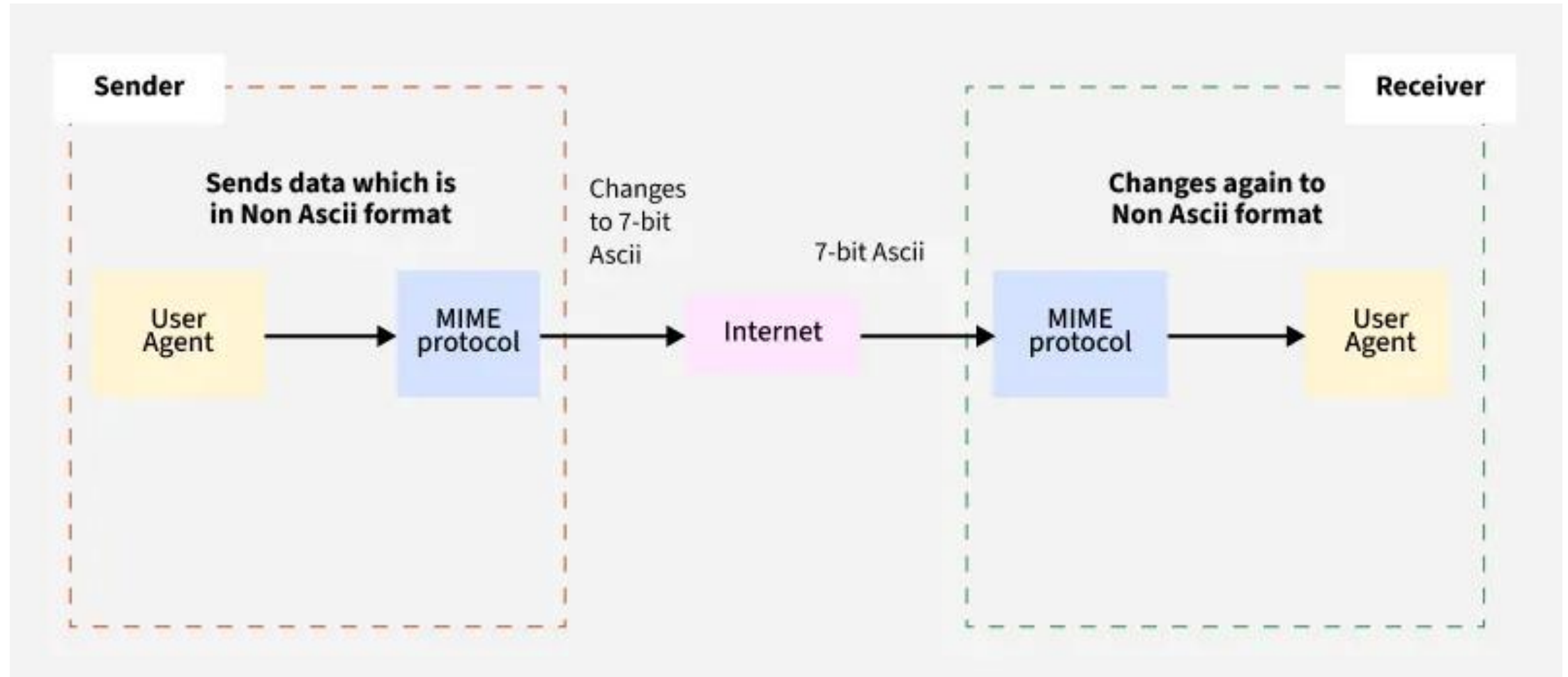
- User Authentication
- Current Working Directory
- Connection State
- Transfer Mode
- File Transfer Parameters



The Importance of MIME in Email Communication



Multipurpose Internet Mail Extension (MIME) Protocol



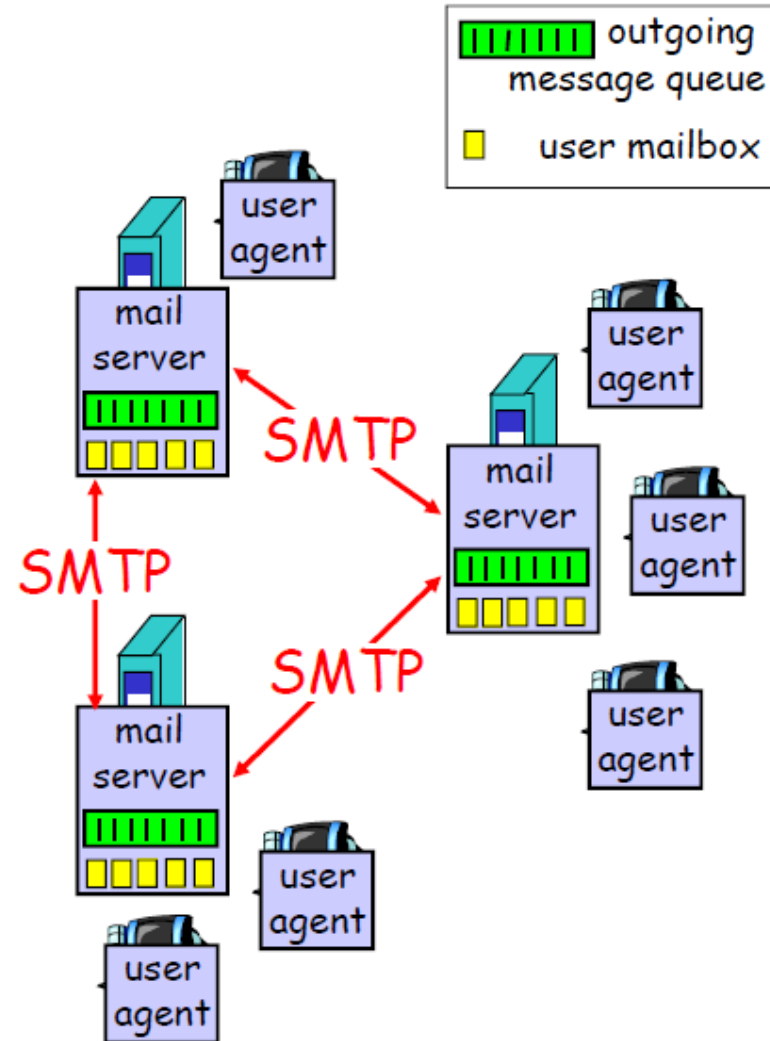
Electronic Mail

Three major components:

- ❖ user agents
- ❖ mail servers
- ❖ simple mail transfer protocol: SMTP

User Agent

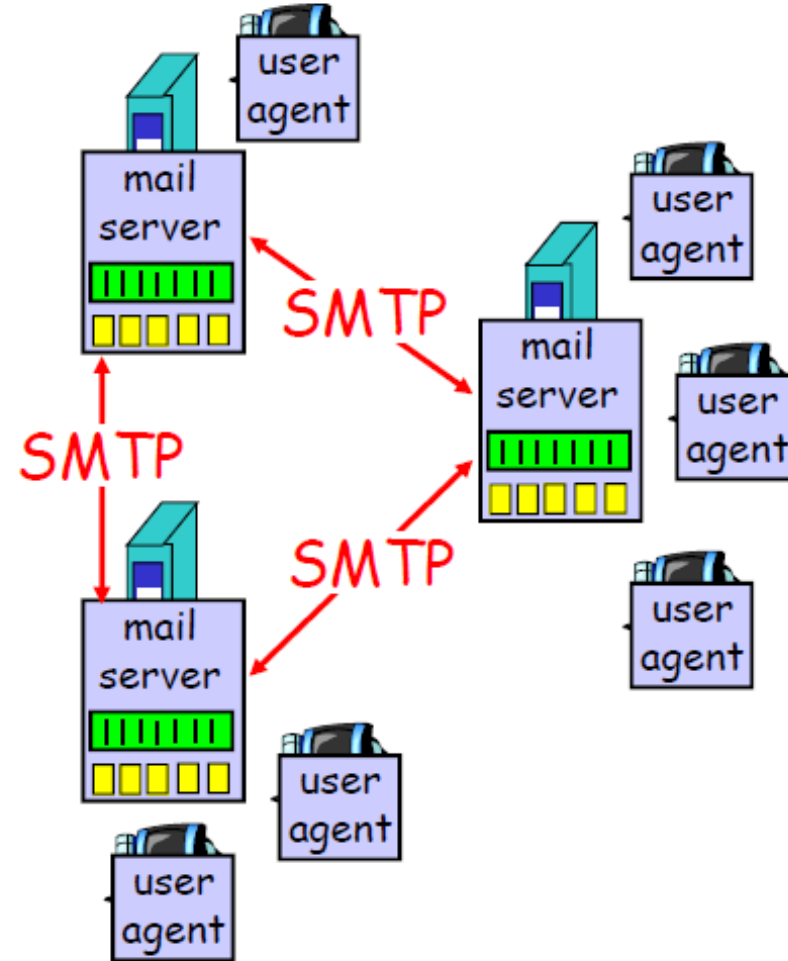
- ❖ a.k.a. "mail reader"
- ❖ composing, editing, reading mail messages
- ❖ e.g., Outlook, elm, Mozilla Thunderbird, iPhone mail client
- ❖ outgoing, incoming messages stored on server



Electronic Mail: mail servers

Mail Servers

- ❖ **mailbox** contains incoming messages for user
- ❖ **message queue** of outgoing (to be sent) mail messages
- ❖ **SMTP protocol** between mail servers to send email messages
 - client: sending mail server
 - "server": receiving mail server

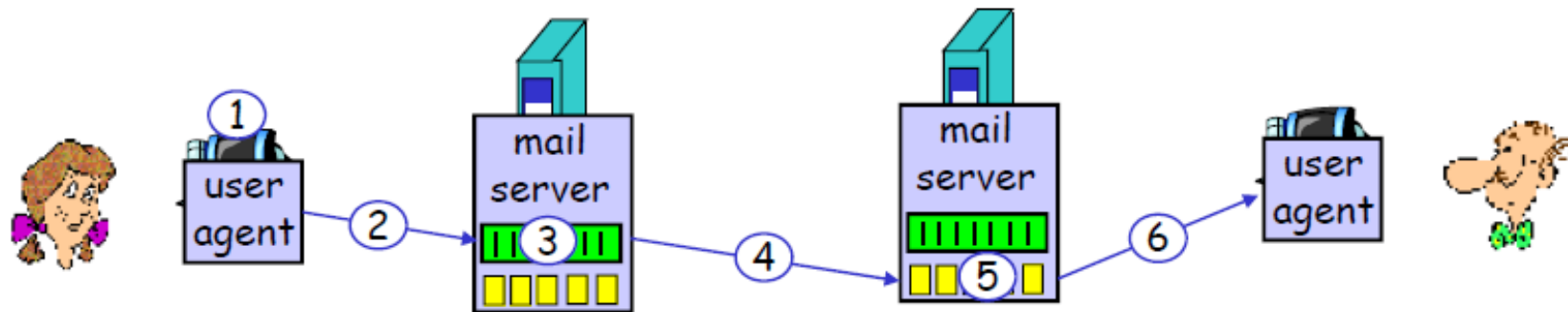


Electronic Mail: SMTP [RFC 2821]

- ❖ uses TCP to reliably transfer email message from client to server, port 25
- ❖ direct transfer: sending server to receiving server
- ❖ three phases of transfer
 - handshaking (greeting)
 - transfer of messages
 - closure
- ❖ command/response interaction
 - **commands:** ASCII text
 - **response:** status code and phrase
- ❖ messages must be in 7-bit ASCII

Scenario: Alice sends message to Bob

- 1) Alice uses UA to compose message and "to" `bob@some school.edu`
- 2) Alice's UA sends message to her mail server; message placed in message queue
- 3) Client side of SMTP opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



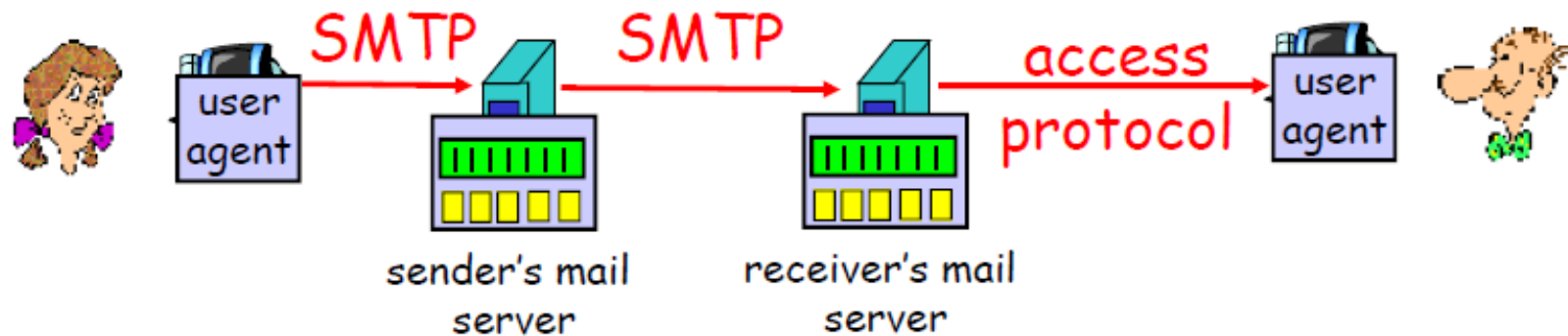
SMTP: final words

- ❖ SMTP uses persistent connections
- ❖ SMTP requires message (header & body) to be in 7-bit ASCII
- ❖ SMTP server uses CRLF.CRLF to determine end of message

comparison with HTTP:

- ❖ HTTP: pull 
- ❖ SMTP: push 
- ❖ both have ASCII command/response interaction, status codes
- ❖ HTTP: each object encapsulated in its own response msg
- ❖ SMTP: multiple objects sent in multipart msg

Mail access protocols



- ❖ SMTP: delivery/storage to receiver's server
- ❖ mail access protocol: retrieval from server
 - POP: Post Office Protocol [RFC 1939]
 - authorization (agent <-->server) and download
 - IMAP: Internet Mail Access Protocol [RFC 1730]
 - more features (more complex)
 - manipulation of stored msgs on server
 - HTTP: gmail, Hotmail, Yahoo! Mail, etc.

DNS: Domain Name System

people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- "name", e.g.,
www.yahoo.com - used by humans

Q: map between IP address and name, and vice versa ?

Domain Name System:

- ❖ *distributed database*
implemented in hierarchy of many *name servers*
- ❖ *application-layer protocol*
host, routers, name servers to communicate to *resolve* names (address/name translation)
 - note: core Internet function, implemented as application-layer protocol
 - complexity at network's "edge"

DNS

DNS services

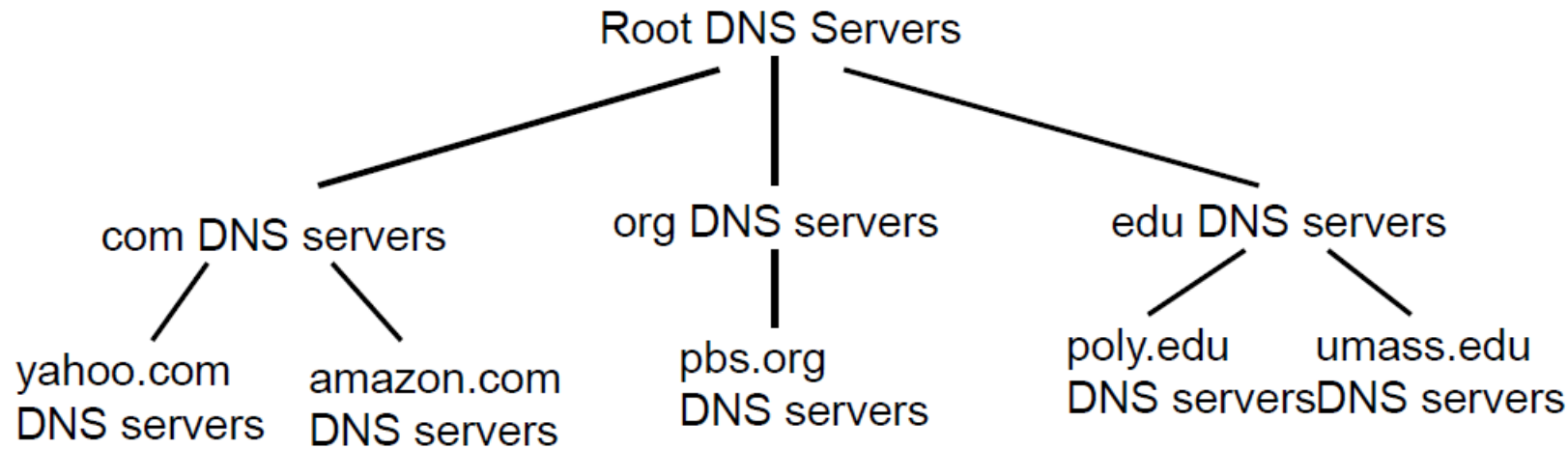
- ❖ hostname to IP address translation
- ❖ host aliasing
 - Canonical, alias names
- ❖ mail server aliasing
- ❖ load distribution
 - replicated Web servers: set of IP addresses for one canonical name

Why not centralize DNS?

- ❖ single point of failure
- ❖ traffic volume
- ❖ distant centralized database
- ❖ maintenance

doesn't *scale!*

Distributed, Hierarchical Database

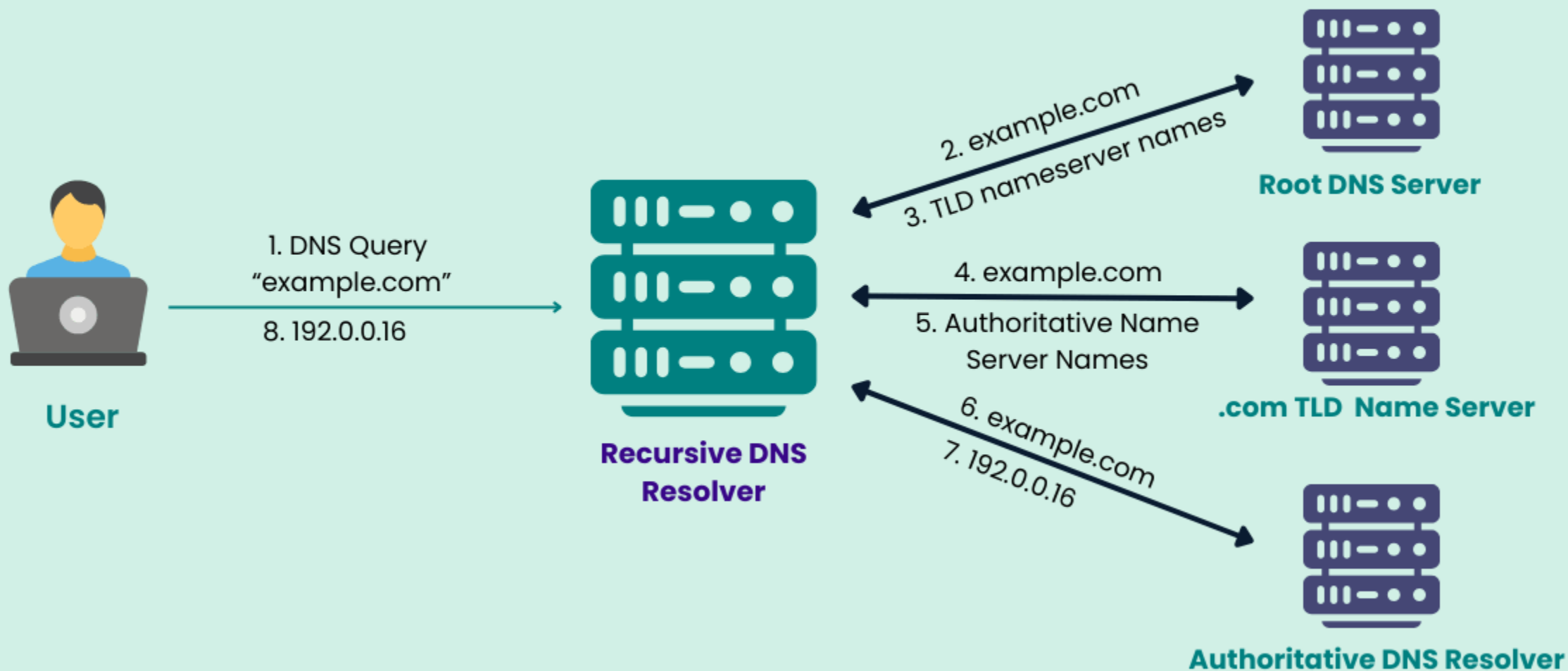


client wants IP for www.amazon.com; 1st approx:

- ❖ client queries a root server to find com DNS server
- ❖ client queries com DNS server to get amazon.com DNS server
- ❖ client queries amazon.com DNS server to get IP address for www.amazon.com

DNS Lookup Process

INDUSFACE™



TLD and Authoritative Servers

Top-level domain (TLD) servers:

- responsible for com, org, net, edu, aero, jobs, museums, and all top-level country domains, e.g.: uk, fr, ca, jp
- Network Solutions maintains servers for com TLD
- Educause for edu TLD

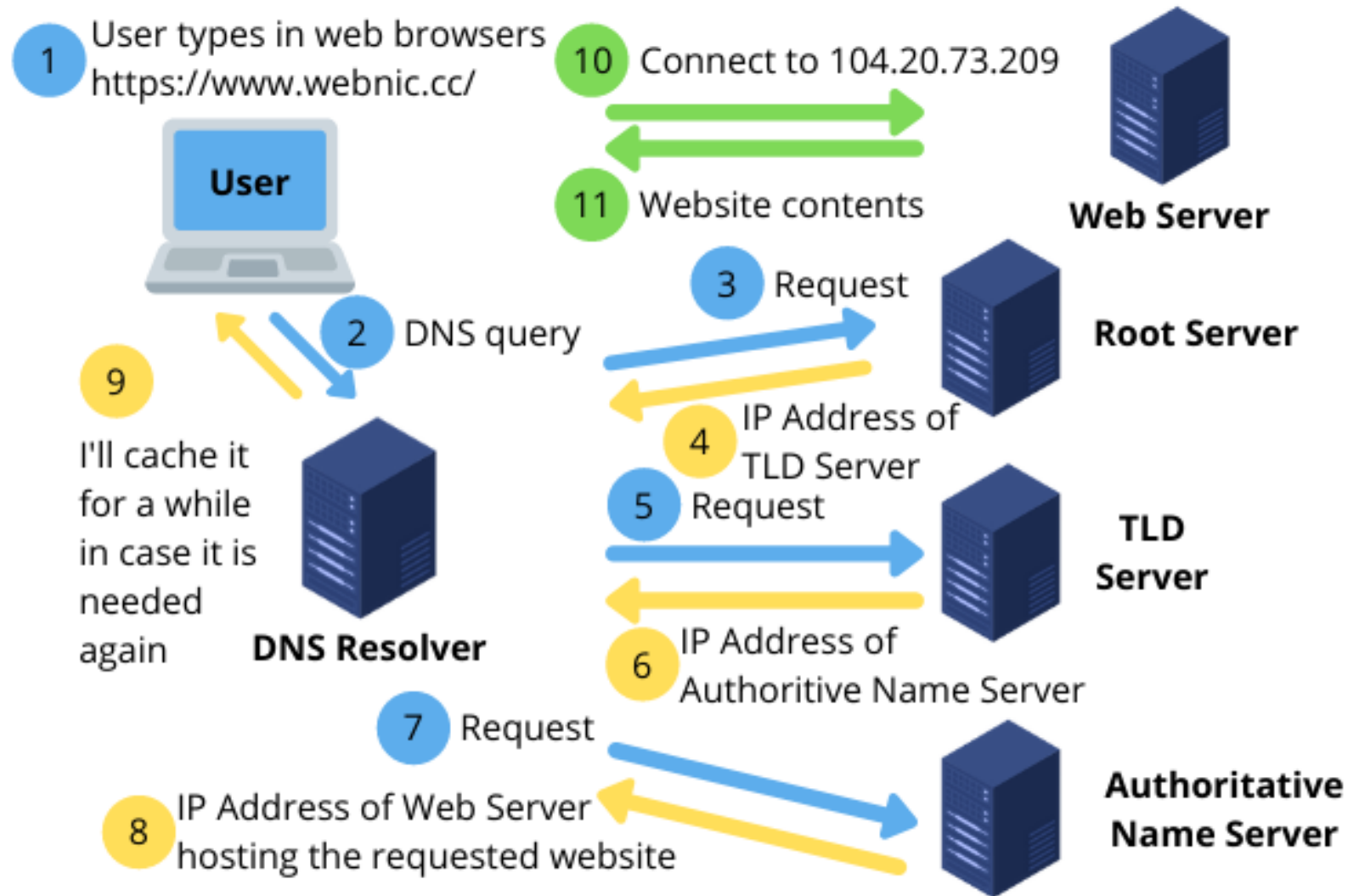
Authoritative DNS servers:

- organization's DNS servers, providing authoritative hostname to IP mappings for organization's servers (e.g., Web, mail).
- can be maintained by organization or service provider

Local Name Server

- ❖ does not strictly belong to hierarchy
- ❖ each ISP (residential ISP, company, university) has one
 - also called "default name server"
- ❖ when host makes DNS query, query is sent to its local DNS server
 - acts as proxy, forwards query into hierarchy

How Does a DNS Work?



TCP		UDP	
FTP	20,21	DNS	53
SSH	22	BooTPS/DHCP	67
Telnet	23	TFTP	69
SMTP	25	NTP	123
DNS	53	SNMP	161
HTTP	80		
POP3	110		
IMAP4	143		
HTTPS	443		

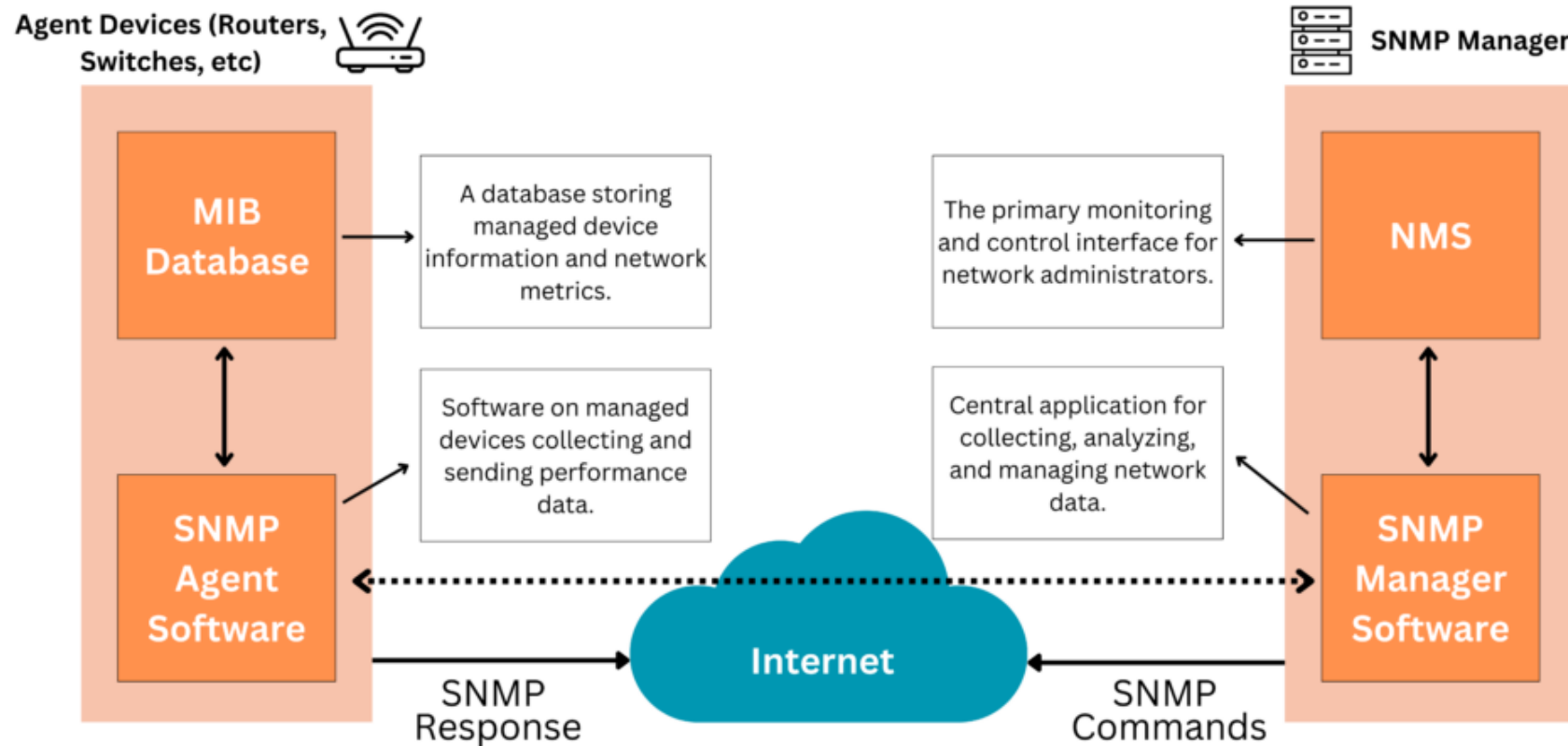
Services & Port Numbers

TCP		UDP	
FTP	20,21	DNS	53
SSH	22	BooTPS/DHCP	67
Telnet	23	TFTP	69
SMTP	25	NTP	123
DNS	53	SNMP	161
HTTP	80		
POP3	110		
IMAP4	143		
HTTPS	443		

Simple Network Management Protocol (SNMP)

Simple Network Management Protocol (SNMP) is an Internet Standard protocol used to manage and monitor network-connected devices in IP networks.

SNMP Architecture



Architecture of SNMP

There are mainly three main components in SNMP architecture:

SNMP Manager

- A centralised system used to monitor the network.
- It is also known as a Network Management Station (NMS).
- A router that runs the SNMP server program is called an agent, while a host that runs the SNMP client program is called a manager.

SNMP agent

- A software module installed on a managed device.
- The manager accesses the values stored in the database, whereas the agent maintains the database.